

I. But du TP

Ce TP a pour objectif de familiariser les étudiants avec le langage de programmation Python. Il présente les concepts de base nécessaires à la réalisation des travaux pratiques à venir. Toutefois, une maîtrise avancée de ce langage requiert plus qu'un simple TP et nécessite une pratique progressive ainsi que des lectures approfondies.

II. Introduction

Python est un langage de programmation interprété (non compilé) et de haut niveau. La première version a été conçue par Guido van Rossum et publiée en 1991. Ses caractéristiques en font l'un des langages les plus utilisés aujourd'hui. Parmi ces caractéristiques, on trouve :

- **Open source et gratuit** : Python est open source, et vous pouvez télécharger son code gratuitement pour l'utiliser et le modifier pour vos projets.
- **Polyvalent** : utilisé dans de nombreux domaines (IA, data science, développement web, etc.),
- **Facile à apprendre** grâce à sa syntaxe claire et lisible.
- **Des bibliothèques étendues** : La bibliothèque standard de Python est très riche. Il existe aussi une large gamme d'API pour divers domaines. Les exemples incluent, NumPy, Pandas, SciPy. etc.
- **Orienté objet** : Python utilise des concepts de programmation orientée objet, tels que les objets, les classes, l'héritage, l'encapsulation des données, etc.
- **Typé dynamiquement** : La spécification d'un type de variable tel que char, int, double, etc., n'est pas nécessaire en Python car elle est décidée pendant l'exécution du code plutôt qu'au départ. Par conséquent, cela rend la programmation plus simple et réduit de nombreuses lignes de code.
- **Portable** : Python utilise un code portable. Ainsi, vous pouvez utiliser le même code sur différentes plateformes comme Windows, macOS, Unix ou Linux.

III. Installation

1. **Installez Python**: il faut évidemment installer la dernière version de Python : <https://www.python.org/downloads/>.

Vérifier l'installation en ouvrant un terminal et en tapant `python --version` ou `python3 --version`.

2. **Installez un gestionnaire de packages Python** (si il n'est pas déjà installé) comme « PIP » : Normalement l'installation de Python installe en même temps le gestionnaire « PIP ». Un gestionnaire de packages est nécessaire pour l'installation des bibliothèques non standards.

Pour vérifier tapez sur l'invite de commande : `pip --version`

3. **Choix d'un éditeur de code**

Bien qu'il soit possible d'utiliser l'éditeur de code par défaut de Python ou un simple éditeur de texte, il est crucial de choisir un éditeur de code (ou IDE : Environnement de Développement Intégré) de haut niveau, offrant des fonctionnalités telles que la coloration syntaxique, l'auto-complétion et des outils de débogage avancés. Parmi les éditeurs et IDE les plus utilisés, on peut citer : **Visual Studio Code** (<https://code.visualstudio.com/download>), **PyCharm** (<https://www.jetbrains.com/pycharm/>), **Sublime Text** (<https://www.sublimetext.com/>), **Jupyter Notebook** (<https://jupyter.org/>), **Spyder** (<https://www.spyder-ide.org/>).

IV. Les bases de Python

- **une Instruction** : chaque instruction occupe une ligne : il n'y a pas de symbole de fin (ex : « ; » comme C ou Java).
- **Blocs d'instruction** : un bloc n'est pas délimité par un symbole particulier ({ et }) mais repérés par l'**indentation** (les espaces) des instructions :

Exemple, un bloc while :

```
while (i<=10):
    print(i)
    i = i + 1
```

- **Les commentaires** :

```
# ce ci est un commentaire sur une ligne
"""
un commentaire sur plusieurs
ligne.
"""
```

L'extension de fichier standard des scripts Python est .py.

- **Affichage** :
l'affichage dans Python se fait en appelant la fonction « print »

Exemple :

```
print("Hello Python!") # affiche Hello Python!
```

- **Valeur introduit pas l'utilisateur** :
introduire une valeur (une chaîne de caractères) se fait par la fonction « input ».

Exemple :

```
valeur=input("donnez un nombre entier:") # l'interpréteur attend une valeur
print("la valeur est:",valeur, type(valeur), int(valeur)+1) # Si on introduit 15, python
affiche : la valeur est: 15 <class 'str'> 16
- type(valeur) : retourne le type de la variable.
- il faut faire la conversion de type pour pouvoir faire un calcul : int(valeur)
```

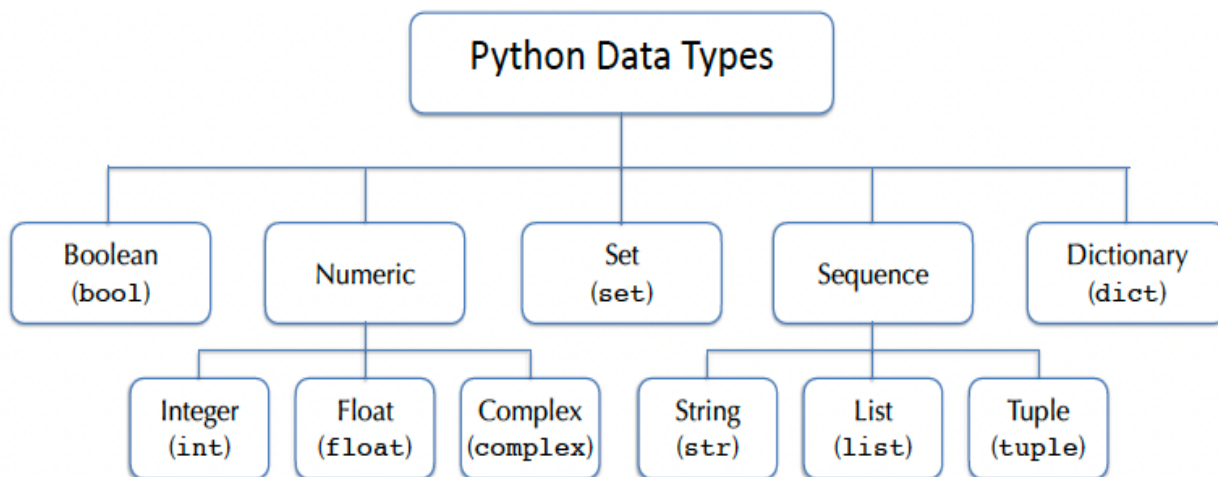
1. Types de base et variables

Python autorise, sans déclaration la manipulation de types classiques : booléens (bool), entiers (int), réels (float), chaîne de caractères (str), listes (list).

Exemple :

```
pi=3.14 # pi est un nombre réel
i=1 # i est un entier
chaine1="This is a string."
chaine2='This is also a string.'
c = '6' # c une chaîne de caractères
r = int(c) * 7 # ... convertie en entier pour calcul
print('réponse: '+str(r)) # ... et retour en chaîne pour affichage
v=True # v est un booléen
```

- Les types par défaut



- Les listes

Les listes sont utilisées pour stocker plusieurs éléments dans une seule variable.

Exemple 1:

```

mylist = [1, 2, 3]
print(mylist) # Affiche: [1, 2, 3]
mylist.append("nombre") # Ajouter un élément à la fin de la liste
x = mylist.pop() # Retirer le dernier élément
mylist.insert(1,4) # insérer 4 à la position 2
print("Valeur retirée:", x) # Affiche Valeur retirée: 4
print(len(mylist)) # Affiche la longueur de la liste
# Parcourir une list (voir boucle for plus loin)
for element in mylist:
    print(element)
  
```

Exemple 2:

```

thislist = ["apple", "banana", "cherry", "orange", "kiwi"]
print(thislist[1]) #Affiche "banana"
print(thislist[-1]) #Affiche "kiwi"
print(thislist[-1]) #Affiche "kiwi"
print(thislist[2:4])# Affiche ['cherry', 'orange']
  
```

- Les Tuples

Un tuple est une collection **ordonnée** et **immuable** (invariable), nous ne pouvons pas modifier, ajouter ou supprimer des éléments après la création d'un tuple. On peut faire des opérations (ajout, suppression, etc.) en passant par des listes.

Exemple :

```

x = ("apple", "banana", "cherry") # déclaration d'un tuple
y = list(x) # création de la liste y a partir du tuple x .
y[1] = "kiwi" # remplacement de « banana » par « kiwi » dans la liste y,
x = tuple(y) # on écrase le tuple x par le tuple tuple(y).
print(x) # affiche ("apple", "kiwi", "cherry")
  
```

- Les ensembles (set)

Les éléments d'un « Set » ne sont **pas ordonnés**, ne **peuvent pas être modifiés** et n'autorisent **pas de valeurs en double**, mais vous pouvez supprimer des éléments et ajouter de nouveaux éléments.

Exemple :

```
set1 = {"apple", "banana", "cherry"} # set 1
set2 = {"pineapple", "banana", "papaya"} # set 2
set1.update(set2) # mettre à jour les éléments de set1 avec set2
print(set1) # affiche {'apple', 'cherry', 'papaya', 'pineapple', 'banana'}
```

- **Les Dictionnaires :**

Les dictionnaires sont utilisés pour stocker des valeurs de données dans des paires « clé:valeur ». Un dictionnaire est une collection ordonnée, modifiable et ne permettant pas les doublons.

Exemple :

```
dict = {
    "brand": "Ford",
    "electric": False,
    "year": 1964,
    "colors": ["red", "blue"]
}
print(dict) # affiche {'brand': 'Ford', 'electric': False, 'year': 1964, 'colors': ['red', 'blue']}
print(dict["colors"]) # affiche ['red', 'blue']
dict["year"] = 2025 # modification de la valeur de la clé "year"
print(dict)
dict["option"] = ["metallic", "rear camera"] # Ajouter 'option': ['metallic', 'rear camera']
dict.pop("option") # Supprime l'item de la clé "option"
dict.popitem() # Supprime le dernier item du dictionnaire
#Parcourir le dictionnaire
for x in dict:
    print(x) # affichage des clés
for x in dict:
    print(dict[x]) # affichage des valeurs
for x in dict.values(): # parcourir par valeur
    print(x)
for x in dict.keys(): # parcourir par clé
    print(x)
for x, y in dict.items(): # parcourir par clé et valeur
    print(x, y)
```

2. Opérateurs :

- **Opérateurs Arithmétiques**

Opérateur	Description	Exemple	Résultat
+	Addition	5 + 2	7
-	Soustraction	5 - 2	3
*	Multiplication	5 * 2	10
/	Division	5 / 2	2.5
//	Division entière	5 // 2	2
%	Modulo (reste)	5 % 2	1
**	Exposant	5 ** 2	25

- Opérateurs de comparaison :

Opérateur	Description	Exemple	Résultat
==	Égal à	5 == 2	False
!=	Différent de	5 != 2	True
<	Inférieur à	5 < 2	False
>	Supérieur à	5 > 2	True
<=	Inférieur ou égal à	5 <= 5	True
>=	Supérieur ou égal à	5 >= 2	True

- Opérateurs Logiques

Opérateur	Description	Exemple	Résultat
and	ET logique	(True and False)	False
or	OU logique	(True or False)	True
not	NON logique	not True	False

- Les opérateurs de chaînes

Python possède deux opérateurs de base de chaîne : l'opérateur de concaténation « + » et l'opérateur de répétition « * ».

Exemple 1:

```
promo1="IA"
promo2="GL"
promoCommun=promo1+" "+promo2
print(promoCommun) #affiche : IA GL
```

Dans python on peut aussi utiliser des variables dans une chaîne de caractères en utilisant l'opérateur « f », dans l'exemple précédent on peut afficher :

```
print(f"promos communs: {promo1} et {promo2}")
```

Exemple 2:

```
prenom="mohamed"
print(3*prenom) #affiche : mohamedmohamedmohamed
```

- Quelques méthode sur les chaîne de caractères

Input	Method	Output
'hello WORLD'	.capitalize()	Hello world
'HELLO WORLD'	.lower()	hello world
'hello world'	.upper()	HELLO WORLD
'Python'	.center(10, '*')	**Python**
'HELLO WORLD'	.count('L')	3
'HELLO WORLD'	.index('O')	4
'HELLO WORLD'	.find('OR')	7
'31/01/2022'	.replace('/', '-')	31-01-2022
'31/01/2022'	.split('/')	['31', '01', '2022']
'abc123'	.isalnum()	True
'12345'	.isnumeric()	True
'hello world'	.islower()	True
'HELLO WORLD'	.isupper()	True

3. Instructions conditionnelles: if, else, elif.

La structure de base est :

if condition:

Code à exécuter si la condition est vraie

else:

Code à exécuter si la condition est fausse

ou :

if condition1:

Code à exécuter si la condition1 est vraie

elif condition2:

Code à exécuter si la condition1 est fausse et condition2 est vraie

else:

Code à exécuter si condition1 et condition2 sont fausses

Exemple :

age = input("Donnez votre age !")

if int(age) >= 18:

print("Vous êtes majeur(e)")

else:

print("Vous êtes mineur(e)")

4. Instructions de boucles :

- **Boucle For :**

Elle peut avoir plusieurs forme :

Exemple 1 :

fruits = ["apple", "banana", "cherry"]

for x **in** fruits:

print(x) *# affiche dans des lignes différents : apple, banana ,cherry*

Exemple 2:

```
for x in "Python":
    print(x) # affiche dans des lignes différents : P puis y , t, h, o et n
```

Exemple 3 :

```
for x in range(6):
    print(x) # affiche dans des lignes différents : 0 1 2 3 4 5
```

Exemple 4 :

```
for x in range(2, 6):
    print(x) # affiche dans des lignes différents : 2 3 4 5
```

● **Boucle while :**

```
while (condition):
    instructions
```

Exemple :

```
i = 1
while i < 6:
    print(i)
    i += 1 # affiche dans des lignes différents : 1 2 3 4 5
```

5. Les Fonctions

En Python, une fonction est définie à l'aide du mot-clé « **def** » :

Exemple : une fonction qui calcule la somme et la produit de deux nombres

```
def somme_produit(a,b):
    return a+b, a*b
#appel de la fonction
s,p=somme_produit(5,6)
print(f"la somme des nombres est {s} et leur produit est {p}")
```

6. Classes et Objets

Python est un langage de programmation orienté objet. Pour créer une classe, on utilise le mot-clé « **class** » :

Exemple : la classe « **Person** » avec deux attributs « **name** » et « **age** » et une méthode qui affiche le nom et l'âge d'un objet de type personne.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def myfunc(self):
        print(f"Hello my name is {self.name} My age is {self.age}")
```

```
p1 = Person("Mohammed", 20) # p1 est une instance de la classe Person
```

```
p1.myfunc() # appel de la méthode "myfunc()
```

- La fonction `__init__` est appelée automatiquement chaque fois que la classe est utilisée pour créer un nouvel objet. C'est le constructeur de la classe.

- Le paramètre « **self** » est une référence à l'instance actuelle de la classe (équivalent à « **this** » de java) .

7. Structures de Données : Piles (Stacks) et Files (Queues)

- Piles (Stacks)

Une liste peut être utilisée comme une pile en utilisant les méthodes **append()** pour ajouter un élément (push) et **pop()** pour retirer un élément (pop).

Exemple :

```
pile = []
# Push (empiler)
pile.append("A")
pile.append("B")
pile.append("C")
print("Pile après empilement :", pile) # Affiche ['A', 'B', 'C']
# Pop (dépiler)
element_depille = pile.pop()
print("Élément dépillé :", element_depille) # Affiche 'C'
print("Pile après dépilement :", pile) # Affiche ['A', 'B']
```

- Files (Queues)

Bien qu'une liste Python puisse être utilisée comme une file, les opérations d'ajout/suppression au début d'une liste sont inefficaces. Pour une file, il est préférable d'utiliser **collections.deque** (double-ended queue), qui est optimisée pour les ajouts et suppressions aux deux extrémités.

Exemple :

```
from collections import deque
file = deque()
# Enqueue (ajouter en fin de file)
file.append("Client 1")
file.append("Client 2")
file.append("Client 3")
print("File après ajout :", file) # Affiche deque(['Client 1', 'Client 2', 'Client 3'])
# Dequeue (retirer en début de file)
client_servi = file.popleft()
print("Client servi :", client_servi) # Affiche 'Client 1'
print("File après service :", file) # Affiche deque(['Client 2', 'Client 3'])
```

Exercices

Exercice 01 :

Écrire le programme Python qui demande à l'utilisateur de saisir un nombre entier et affiche si ce nombre est pair ou impair.

Exercice 02 :

Écrire une fonction récursive `fibonacci(n)` qui calcule le n-ième terme de la suite de Fibonacci. Afficher les 10 premiers termes. La suite de Fibonacci est définie par :

- $F(0) = 0, F(1) = 1$
- $F(n) = F(n-1) + F(n-2)$ pour $n \geq 2$.

Exercice 03 :

Écrire une fonction `analyse_liste(liste)` qui prend en entrée une liste de notes et retourne :

- le minimum,
- le maximum,
- la moyenne,
- une liste des notes supérieurs à la moyenne.

On utilise les fonctions de liste intégrées `min`, `max`, `sum` et `len`.

Exercice 04:

Écrire une fonction Python « `manipuler_liste` » qui prend en paramètre une liste de chaînes de caractères et effectue les opérations suivantes :

1. Ajouter une nouvelle chaîne de caractères à la fin de la liste (en utilisant `append`).
2. Ajouter une nouvelle chaîne de caractères au début de la liste (en utilisant `insert`).
3. Supprimer un élément de la liste en spécifiant son index.
4. Trier la liste par ordre alphabétique croissant.
5. Inverser l'ordre des éléments de la liste.
6. Utiliser la méthode `pop()` pour supprimer les éléments de la liste un par un, jusqu'à ce qu'il ne reste plus que deux éléments. À chaque suppression, afficher un message indiquant la chaîne supprimée.

Exercice 05:

Écrire un programme qui prend une phrase saisie par l'utilisateur et qui :

1. Compte le nombre de mots (Une phrase peut être découpée en mots avec `split()`).
2. Affiche les mots sans doublons (`set`).
3. Crée un dictionnaire associant chaque mot à son nombre d'occurrences.

Par exemple, si l'utilisateur entre la phrase: les enfants aiment jouer avec les autres enfant

Le programme affiche :

Nombre de mots : 8

Mots sans doublons : {'enfants', 'jouer', 'aiment', 'les', 'avec', 'autres'}

Occurrences des mots : {'les': 2, 'enfants': 2, 'aiment': 1, 'jouer': 1, 'avec': 1, 'autres': 1}

Exercice 06

Créez un dictionnaire `capitale_pays` associant des pays à leurs capitales (ex: "Algérie": "Alger", "France": "Paris", "Allemagne": "Berlin", "Japon": "Tokyo").

1. Affichez la capitale de l'Algérie,
2. Tentez d'accéder à la capitale de l'Italie en utilisant `get()` et une valeur par défaut si la clé n'existe pas.
3. Ajoutez un nouveau pays et sa capitale à `capitale_pays` (ex: "Espagne": "Madrid").
4. Modifiez la capitale du Japon en "Kyoto" (pour l'exercice).
5. Supprimez la France du dictionnaire.
6. Parcourez le dictionnaire `capitale_pays` et affichez chaque pays et sa capitale sous la forme "La capitale de [Pays] est [Capitale]".

Exercice 07

Écrivez une fonction `inverser_chaine(chaine)` qui prend une chaîne de caractères en entrée et affiche l'inverse de cette chaîne.

- Utilisez une pile pour inverser la chaîne (empilez chaque caractère, puis dépilez-les pour reconstruire la chaîne inversée).
- Testez votre fonction avec "Bonjour" et "Python".

Exercice 08:

Écrivez une fonction `gerer_taches(liste_taches)` qui prend une liste de tâches en entrée.

- Ajoutez toutes les tâches à une file.
- Tant que la file n'est pas vide, retirez une tâche, affichez "Traitement de la tâche : [Nom de la tâche]", puis "Tâche [Nom de la tâche] terminée."
- Testez votre fonction avec ["Nettoyer", "Coder", "Manger", "Dormir"].

Exercice 09:

Créer une classe `Etudiant` avec les attributs : nom, prenom, note.

- Le constructeur `__init__` initialise les attributs.
- Ajouter une méthode `mention()` qui renvoie :
 - "Passable" si $\text{note} \geq 10$
 - "Assez bien" si $\text{note} \geq 12$
 - "Bien" si $\text{note} \geq 14$
 - "Très bien" si $\text{note} \geq 16$
- La méthode `afficher` affiche les informations de l'étudiant.
- Créer une liste d'étudiants et afficher ceux qui ont "Bien" ou plus.

Enfin, implémentez une fonction `rechercher_etudiant(list_etudiant, nom_etudiant)` qui cherche dans la liste précédente les étudiants qui ont le nom « `nom_etudiant` ».